Robustness of Text Classification to Adversarial Unicode Perturbations

Lorenzo Marinelli (2043092)
Sapienza University of Rome
Natural Language Processing, 2025–2026

Abstract

A sentence can look almost unchanged yet become difficult for a text classifier to read. This project tests that gap using Unicode substitutions and insertions on BANKING77 and TREC. Three TF–IDF classifiers isolate the role of word, character and BPE features; fine-tuned DistilBERT provides a pretrained reference. Character features retain the most performance at the primary 20% attack strength, while DistilBERT's robustness depends strongly on the attack family. Simple preprocessing recovers much of the lost performance, and a restricted confusable map almost restores the clean scores within the experiment's restricted attack set. These findings support the usefulness of targeted input handling, with clear limits on generalization beyond English-only use.

1 Task description/Problem statement

To an average reader, Latin `a` and its Cyrillic lookalike will appear identical. To a machine learning classifier, their code points U+0061 and U+0430 are different inputs. Invisible characters create the same problem without replacing a visible letter. A familiar word can lose its vocabulary entry or split into unusual subword tokens even when its meaning remains easy to recognize to a human reader.

I studied how this mismatch affects short-text classification and whether inexpensive preprocessing can repair it. The central comparison uses the same Logistic Regression approach and changes its representation: word, character or BPE TF–IDF. DistilBERT adds a contextual, pretrained comparison. Every system receives the same perturbed examples.

The analysis follows three questions: how quickly performance falls as the attack budget grows, which transformations cause the largest failures, and how much of the lost performance input repair can recover.

The task is to assign one label to a short input text: a banking intent for BANKING77 or an answer type for TREC. After a Unicode attack, the expected label stays the same. I check whether the classifier still predicts it and whether simple input repair helps.

1.1 Examples

These are two real inputs from the test sets:

- BANKING77: "How do I locate my card?" → `card_arrival`.
- TREC: "How far is it from Denver to Aspen ?" → `NUM`, a numerical answer.

1.2 Real-world applications

Intent classification can route a banking request to the right support flow. Question classification can guide a question-answering system toward a person, place, number or another answer type. Unicode changes can also matter in content moderation, where text may be deliberately obfuscated. I focus on the first two tasks here; they let me compare robustness without changing the intended meaning of an example.

2 Related work

2.1 From visible substitutions to encoding attacks

Eger et al. [1] studied visual character substitutions across character, word and sentence level NLP tasks. Both neural and non-neural systems were sensitive to changes that human readers could often interpret. The three shielding approaches they tested were visual character embeddings, adversarial training and rule-based recovery. Those defenses improved robustness but did not fully restore clean performance.

Boucher et al. [2] broadened the problem to encoding-level attacks: invisible characters, homoglyphs, reordering controls and deletion controls. They demonstrated black-box failures across several NLP applications, including deployed systems. I studied a narrower setting with fixed perturbation budgets and the same attacked examples for every classifier. Reordering and deletion controls are outside this experiment.

2.2 Why tokenization matters

Geh et al. [3] investigated a different interface. They selected alternative valid tokenizations of an unchanged string to bypass language-model safety restrictions. Here, the tokenizer is fixed and the Unicode input changes. These attacks should not be treated as equivalent, but both make the token sequence relevant to robustness.

2.3 What a repair method can recover

Cooper et al. [4] used language models to normalize homoglyph-obfuscated tweets, either by directly replacing lookalikes or by reconstructing text after non-Latin characters were masked. Their approach can address substitutions beyond a fixed repair dictionary.

3 Datasets and benchmarks

BANKING77 tests fine-grained banking intent detection [5]; TREC tests six coarse question categories [6]. I retained both official test sets and removed training texts also present in the test set, then reserved 10% of the remaining training data for stratified validation (split seed 42).

Table 1: Data partitions.

Dataset	Classes	Train	Valid.	Test
BANKING77	77	9,002	1,001	3,080
TREC	6	4,897	545	500

The BANKING77 release contains 10,003 training queries and 3,080 test queries across 77 intents. The TREC question-classification release provides training and test questions with coarse and fine labels; I use only its six coarse categories. BANKING77 tests distinctions within one domain, while TREC tests broader answer types. Both are short English texts, so this comparison does not cover long documents or multilingual input.

For a related robustness dataset, Cooper et al. [4] augmented 29,846 tweets with homoglyph substitutions observed in social media. Their setting is closer to obfuscated moderation inputs. Here I generate attacks on the two existing classification test sets, keeping each clean example and its ground truth (gt) label for paired evaluation.

4 Existing tools, libraries, papers with code

I used scikit-learn for TF–IDF, Logistic Regression and scoring; tiktoken for the `o200k_base` BPE features; and Hugging Face Transformers with PyTorch for DistilBERT. The pretrained checkpoint is `distilbert-base-uncased`. Python's `unicodedata` provides normalization. The lock file records the package versions used by the notebooks.

Published implementations provide useful reference points: Boucher et al. release their attack code and experiments, and Cooper et al. provide homoglyph augmentation code and normalization prompts. The TREC authors also publish preprocessing examples and feature scripts alongside the data. I implemented the restricted attacks and deterministic defenses directly in the notebooks so that their behavior is easy to inspect.

Notebooks 00–06 contain the local experiments; notebook 07 contains the "last minute" Jev comparison. The submission includes the saved metric tables, figures, report source and compiled PDF. Datasets, fitted checkpoints and prediction caches must be recreated or supplied separately when rerunning the notebooks.

5 State-of-the-art evaluation

The relevant papers evaluate different tasks and attack settings, so their results do not form one directly comparable ranking. Eger et al. [1] compare task performance before and after visual substitutions and test shielding at increasing perturbation levels. Boucher et al. [2] use black-box attacks across several NLP applications and examine how a small number of encoding changes affects the output. Cooper et al. [4] evaluate both text normalization and downstream hate-speech classification after repair.

For clean intent detection, Casanueva et al. [5] report accuracy with limited examples per intent and with the full training set. That answers a different question from robustness to Unicode changes. My comparison uses fixed attacked test sets, accuracy and Macro-F1, clean-to-attacked loss, attack

success and defense recovery.

6 Comparative evaluation

6.1 The four different text representations

The classical models share Logistic Regression, the data split and validation. Vocabulary sizes and feature geometry still differ because the representations work differently: for example, a single word produces several character n-grams. DistilBERT [7] has a different architecture, so it serves as a separate reference.

Table 2: Model input representations.

Model	Input representation
Word TF–IDF	Word unigrams and bigrams.
Character TF–IDF	Character 3–5-grams.
BPE TF–IDF	`o200k_base` token-ID unigrams and bigrams.
DistilBERT	Uncased WordPiece; maximum 128 tokens.

For all three TF–IDF models, I chose $C \in \{0.1, 1, 10\}$ by validation Macro-F1; each selected $C = 10$ on both datasets. DistilBERT used three epochs, a learning rate of 2×10^{-5}, effective batch size 16, weight decay 0.01 and warmup ratio 0.10.

6.2 What changes in an attacked input?

Each attack samples eligible positions reproducibly, then the clean and attacked examples gets stored together with the edits made.

Table 3: Attack families and where they apply.

Family	Transformation and eligibility
Homoglyph	Replace mapped letters with curated Greek or Cyrillic lookalikes.
Invisible	Insert a default-ignorable control after an alphanumeric character.
Compatibility	Use alternate forms whose NFKC normalization recovers the source.
Composition	Decompose a precomposed character when it has a canonical decomposition.
Mixed	At each eligible position, sample one applicable transformation.

The attack strengths tested are 5%, 10%, 20%, 40% and 75% of eligible positions.

6.3 Coverage of the methods

A nominal budget is meaningful only if the attack can effectively change the text samples. The four broad families reach every test example at 20% (I'll take this as the main strength from now on); canonical composition reaches none (Table 4). Its unchanged test score therefore provides no evidence of robustness, because no test input has an eligible canonical decomposition. Naturally accented text could give a different result.

Table 4: Coverage and average edits per example at 20% strength. B77 denotes BANKING77.

	B77		TREC	
Attack	Cover.	Edits	Cover.	Edits
Homoglyph	100%	4.86	100%	3.26
Invisible	100%	8.54	100%	5.64
Compatibility	100%	8.85	100%	5.88
Composition	0%	0.00	0%	0.00
Mixed	100%	8.85	100%	5.88

Budgets are relative to each family's eligible positions so the same percentage does not produce the same number of edits for different attack kinds.

6.4 Five test-time defenses

The models are evaluated without retraining. Each wrapper processes both clean and attacked inputs, making any clean-text harm visible. NFC and NFKC follow Unicode normalization rules; neither is a general solution to cross-script lookalikes [8, 9].

6.5 How the result measures work

Let N be the number of test examples and K the number of classes. For class k, TP_k counts correct predictions of that class, FP_k counts incorrect predictions of it, and FN_k counts its missed examples.

Accuracy. Correct predictions divided by N; each example has equal weight.

Macro-F1. Precision and recall are balanced within each class, then all classes have equal weight:

$$M = \frac{1}{K} \sum_{k=1}^{K} \frac{2TP_k}{2TP_k + FP_k + FN_k}.$$

From now on, M denotes Macro-F1. A class with no correct predictions receives F1 zero. Accuracy weights frequent classes more heavily, while Macro-F1 prevents them from dominating the average. That's why it is useful in these multiclass classification tasks.

Macro-F1 loss. $M_{\text{raw,clean}} - M_{\text{raw,attacked}}$. A loss of 0.10 is ten F1 percentage points, not a 10% relative loss.

Attack success rate (ASR). Newly incorrect examples divided by examples correct when clean. It

Table 5: Defense scope. "Full repair" names the most complete wrapper in this experiment.

Wrapper	Operation	Intended coverage
NFC	Canonical composition.	Canonical equivalence; untested here because composition coverage is zero.
NFKC	Compatibility normalization.	The generated compatibility substitutions.
Control removal	Remove default-ignorable characters.	The selected invisible insertions.
NFKC + removal	Apply both operations in sequence.	Compatibility and invisible changes.
Full repair	Add restricted non-ASCII-to-ASCII confusable repair.	Also the curated homoglyphs; assumes English/ASCII targets.

measures how often an attack overturns an initially correct prediction; baseline errors are excluded. ASR is undefined if no examples are correct when clean. For example: if 80 of 100 examples are correct when clean and an attack overturns 20 of those 80 predictions, ASR is $20/80 = 25\%$. It is not the attacked error rate: the 20 original errors are outside its denominator, and some of those could become correct after attack (which would be by sheer luck and it's not worth investigating in the current study).

Clean-score delta. $M_{\text{defended,clean}} - M_{\text{raw,clean}}$. A negative value means that the wrapper harms clean performance.

6.6 Defense recovery R

Recovery measures the fraction of the attack-induced Macro-F1 loss that a wrapper restores:

$$R = \frac{M_{\text{defended,attacked}} - M_{\text{raw,attacked}}}{M_{\text{raw,clean}} - M_{\text{raw,attacked}}}. \quad (1)$$

If $R = 0$, the attacked score is unchanged; $R = 0.5$ means half of the lost Macro-F1 is recovered; and $R = 1$ means the clean baseline is reached. Values are not clipped so repair can make the score worse ($R < 0$) or raise it above the clean baseline ($R > 1$), lastly Recovery is undefined when the raw attack causes no positive loss.

For each condition, I calculate the metrics on the matched test examples, but to keep tidyness and avoid entire pages of only tables for some of them I averaged the scores of the different attacks families. The overall attacked score gives equal weight to homoglyph, invisible, compatibility and mixed attacks. Composition is excluded from that average because it changes no inputs.

6.7 Results

To keep the presentation clear, tables and plots show mean scores without standard-deviation annotations.

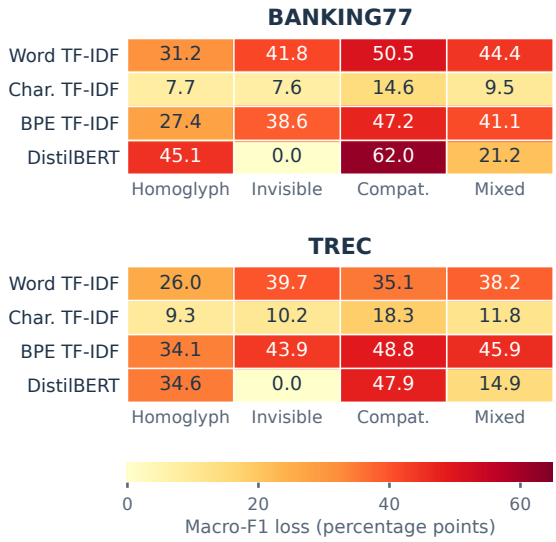


Figure 1: Macro-F1 loss at 20% strength, without defenses (larger is worse). Mixed is not always the hardest attack: on BANKING77, DistilBERT loses 62.0 points to compatibility changes, 21.2 to mixed attacks and none to invisible controls. Composition is omitted because it changes no inputs.

Clean accuracy does not settle the robustness comparison. Character TF–IDF has the strongest clean BANKING77 score, while DistilBERT leads on TREC. At the main 20% strength setting, character features retain the highest average Macro-F1 on both datasets (Table 6). Their attack-success rates are also the lowest: fewer initially correct predictions are overturned.

Table 6: Clean and attacked test performance. Attacked F1, loss and ASR average the four effective families at 20%.

Model	Clean acc.	Clean F1	Attacked F1	F1 loss	ASR
BANKING77					
Word TF-IDF	0.885	0.886	0.466	0.420	49.0%
Character TF-IDF	0.906	0.906	0.807	0.099	12.5%
BPE TF-IDF	0.882	0.882	0.496	0.386	45.3%
DistilBERT	0.832	0.819	0.498	0.321	40.8%
TREC					
Word TF-IDF	0.888	0.885	0.537	0.347	38.6%
Character TF-IDF	0.888	0.883	0.759	0.124	17.9%
BPE TF-IDF	0.890	0.879	0.447	0.432	45.4%
DistilBERT	0.967	0.944	0.701	0.243	27.4%

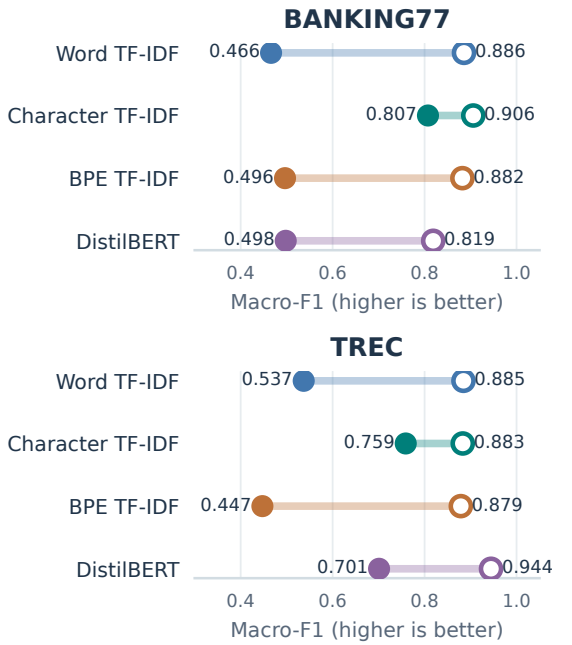


Figure 2: The main result at a glance. Filled points show averaged Macro-F1 at 20% perturbation, hollow points show clean performance.

Performance across the full attack-strength range. Figure 3 follows all measured strengths. Mean Macro-F1 decreases and attack success rises as more eligible positions are changed. Character features lead on BANKING77 throughout. On TREC, DistilBERT leads at 5% (0.865 versus 0.853); character features lead from 10% to 40%, but DistilBERT leads again at 75% (0.411 versus 0.337). All four models are evaluated through 75%.

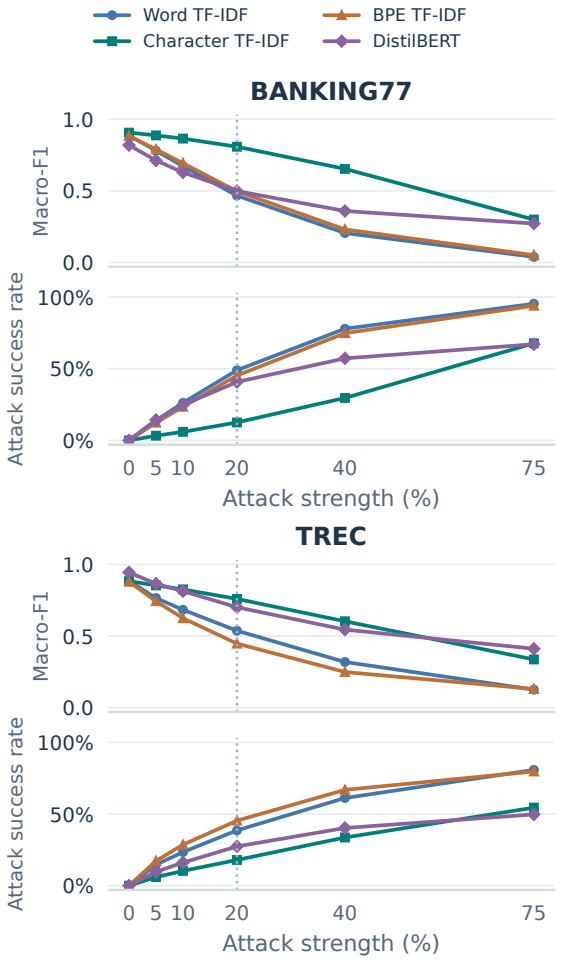


Figure 3: Macro-F1 (higher is better) and attack success rate (lower is better), averaged equally across four effective families. Zero denotes clean text; the dotted line marks 20%.

What remains at 40% and 75%?. The stronger attacks reveal substantial losses even for the most robust representation (Figure 4). On BANKING77, character Macro-F1 falls from 0.653 at 40% to 0.299 at 75%, while word and BPE features reach 0.039 and 0.051. DistilBERT reaches 0.271. At 75%, attacks overturn 67.8% of the pre-

dictions that the character model got right when clean.

On TREC, character Macro-F1 falls from 0.603 at 40% to 0.337 at 75%, while word and BPE features reach 0.126 and 0.130. DistilBERT reaches 0.411. At 75%, attacks overturn 54.4% of the predictions that the character model got right when clean.

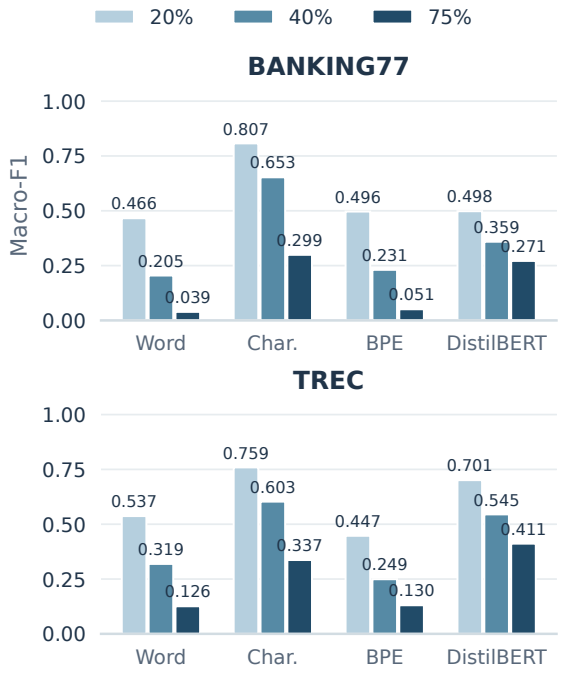


Figure 4: Mean Macro-F1 at 20%, 40% and 75%, using the same four-family average.

The attack family changes the trajectory. The family breakdown in Figure 5 explains why the mean alone is incomplete. DistilBERT stays at its clean score under the tested invisible controls through 75%, while its BANKING77 Macro-F1 heavily suffers under compatibility substitutions . Its preprocessing discards the tested invisible characters. Character features also degrade: compatibility substitutions give them the lowest 75% score on both datasets. Mixed attacks sample applicable transformations and as such will not necessarily perform like the worst single family.

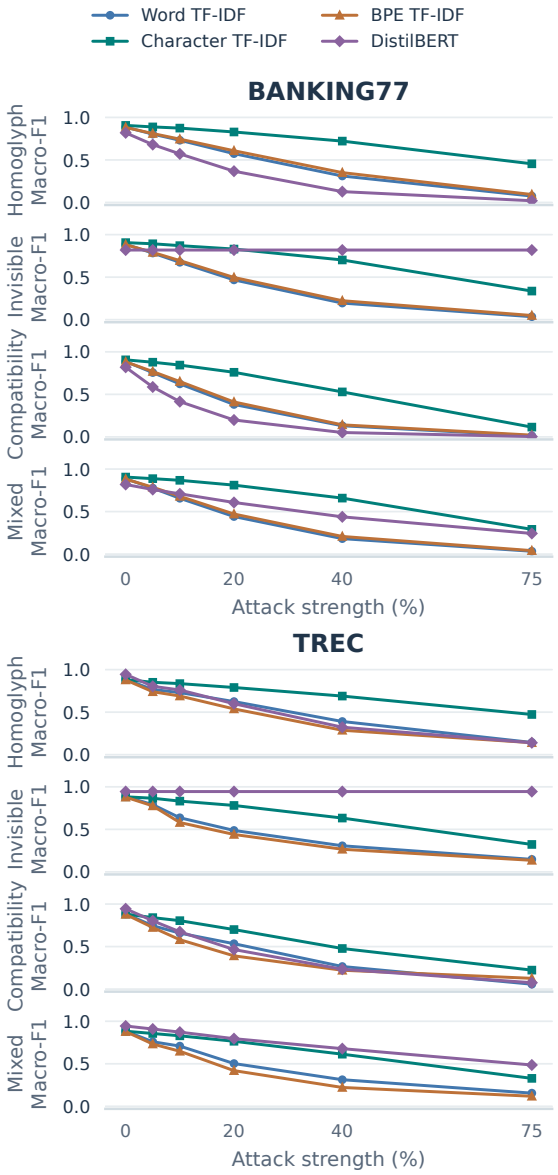


Figure 5: Macro-F1 by attack family and dataset across all available strengths. Each curve begins at its clean score. Composition is omitted because it changes no test inputs. All four models cover the full range through 75%.

Table 7: Mean defense recovery at 20% strength, shown as $100R$: 72.31 means 72.31% of the lost score recovered on the equal-family average. Each classical row averages four defined family ratios. DistilBERT averages three, excluding invisible attacks because their raw score loss is zero. Rounded 100.00 values need not imply exact recovery.

Model	NFC	NFKC	Removal	NFKC + removal	Full repair
BANKING77					
Word TF-IDF	0.00	28.66	42.96	72.31	99.98
Character TF-IDF	0.00	31.00	42.67	72.29	100.00
BPE TF-IDF	0.00	29.18	43.13	72.43	99.98
DistilBERT	0.00	54.42	0.00	54.42	99.99
TREC					
Word TF-IDF	0.00	28.17	42.67	72.06	100.00
Character TF-IDF	0.00	32.28	40.21	70.86	100.00
BPE TF-IDF	0.00	28.27	41.63	70.98	99.99
DistilBERT	0.00	51.97	0.00	51.97	100.00

How much of the lost score is recovered?.

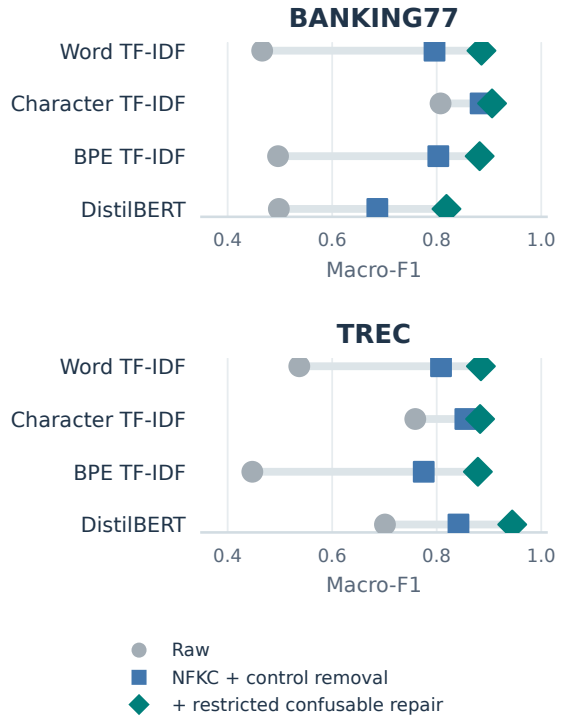


Figure 6: Macro-F1 before and after repair, averaged across the four effective families at 20%. Full repair nearly reaches the clean baseline; its map covers the restricted attack whitelist.

Combining NFKC and control removal improves classical recovery because they repair different families. DistilBERT already discards the tested controls, so removal adds nothing. Its combined recovery is 54.42% on BANKING77 and 51.97% on TREC over three attack families (not the homoglyph).

A worked recovery calculation. For BANKING77 word TF–IDF under the 20% mixed attack, clean Macro-F1 is 0.885686, raw attacked Macro-F1 is 0.441687, and NFKC plus removal raises it to 0.837951. Applying Equation 1 gives:

$$R_{\text{mixed}} = \frac{0.837951 - 0.441687}{0.885686 - 0.441687} \approx 0.892.$$

The defense recovers 89.2% of this loss. Its four-family mean is 72.31%: compatibility and invisible attacks are fully repaired, while homoglyphs are not. Calculating R from cross-family average scores would weight families by their initial loss; the table instead averages their recovery ratios equally. Near-complete recovery is measured on this restricted attack set.

6.8 Two examples behind the scores

I selected as an example 2 dataset/model pair with very mixed-attack ASR at 20%, then looked at its attacks across families. The largest successful probability loss (resulting in missclassification) and largest repair gain identify the same sentence, shown in (Table 8). The second example instead has the largest loss among resisted attacks(so correct classification even under attack) (Table 9).

The gt-probability is the probability assigned to the ground truth (gt) label.

Lookalikes overturn the first prediction; repair restores it. In the second case, confidence falls but the correct label stays first. Every non-ASCII code point is exposed in the audit string as they would be indistinguishable (else there would be no point in this whole study).

Failure and recovery / homoglyph / 5 edits	
Clean text	There is a suspicious direct debit payment.
Code-point audit	`T[U+04BB]ere is a suspi[U+0441][U+0456]ous dire[U+0441]t deb[U+0456]t payment.`
gt-probability	Clean 0.999 → attacked 0.001 → repaired 0.999

Table 8: BANKING77, Word TF-IDF, 20% strength, attack seed 13.

Prediction retained / invisible / 5 edits	
Clean text	Do you offer Visa or Mastercard?
Code-point audit	`Do yo[U+200B]u of[U+200B]fer Vi[U+200D]s[U+2060]a or Mastercar[U+200B]d?`
gt-probability	Clean 0.998 → attacked 0.042 → repaired 0.998

Table 9: BANKING77, Word TF-IDF, 20% strength, attack seed 13.

6.9 Extra comparison with Jev

As I was working on the previous part of this homework, I was looking at the eventual comparison I could make with a "frontier" model. The initial idea was to use an LLM through an API, but in those days conveniently the hype for Jev exploded, a "new" (not really new) paradigm which uses a System One architecture: it returns typed decisions and probabilities [10]. Earlier related work includes the March 2025 SalesRLAgent paper on reinforcement learning for sales-conversion probabilities [11], by the author who later developed Laya, which has public weights [12].

I conducted an additional zero-shot comparison using Jev's hosted classification API. Notebook 07 pins `jev-1.13.0` and asks one fixed *Choice* question per input [13, 14]. BANKING77 supplies all 77 intent names with readable descriptions; TREC supplies six answer-type descriptions. The input text and question go to the API while the ground truth (gt) label stays local for evaluation.

The test covers all 3,080 BANKING77 and 500 TREC examples, clean and under four working attack families at 5%, 10%, 20%, 40% and 75% strength. Each attack family has equal weight in Table 10; Figure 7 shows them separately.

Table 10: Jev on clean and attacked text. Attacked scores and ASR average the four families equally. F1 loss is in percentage points; ASR is a percentage.

Strength	Accuracy	Macro-F1	F1 loss	ASR
BANKING77				
Clean	0.801	0.793	–	–
5%	0.798	0.791	0.3	1.3
10%	0.796	0.789	0.4	1.6
20%	0.795	0.787	0.6	1.9
40%	0.792	0.785	0.9	2.5
75%	0.787	0.780	1.3	3.3
TREC				
Clean	0.944	0.939	–	–
5%	0.942	0.937	0.2	0.5
10%	0.943	0.937	0.3	0.5
20%	0.946	0.942	-0.2	0.5
40%	0.940	0.934	0.5	0.7
75%	0.943	0.941	-0.2	0.6

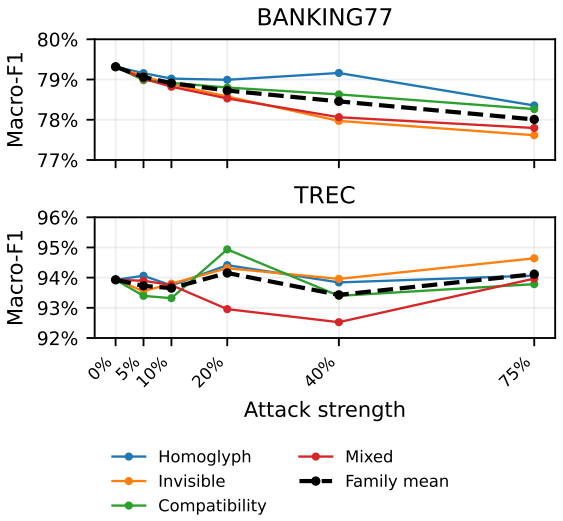


Figure 7: Jev's Macro-F1 by attack family and their equal-weight mean (black dashed). Both panels start at clean text and use different vertical ranges to show the small changes.

On BANKING77, mean Macro-F1 falls gradually from 0.793 on clean text to 0.780 at 75% strength. Invisible characters cause the largest loss there (1.7 percentage points). On TREC, the scores fluctuate around the clean result (0.939), with no steady decline as strength increases; the mean at 75% is 0.941. Small gains can even occur when increasing strength when a new perturbation corrects some clean mistakes.

This is to be considered an exploratory comparison of a new system with different training and capacity. As much as the robustness of Jev in this study may look phenomenal compared to the more classical approaches, we can't be absolutely confident in it because of the lacking knowledge we have regarding the training data used for the model and all the surrounding infrastructure around the API. An explanation for these performances is that there could be normalization methods applied to the input similar to the defenses used in this study.

6.10 Discussion

The main takeaway that can be taken from these experiments is that good performance on clean text can hide a dependence on its exact spelling and encoding. Character features generally tolerate the tested changes better than word and BPE features, probably because of their higher "density" but they still struggle when much of the input is modified. DistilBERT also shows how different the outcome can depend on the attack: its preprocessing removes the tested invisible characters, while other substitutions remain damaging. A single average score would hide these differences.

The defense results show how much can be recovered by changing the way the input is handled, without retraining the classifier. This makes simple preprocessing worth considering before moving to a more complex model. The almost complete recovery with full repair needs some caution, though: the repair map covers the characters used by the attacks. I would expect a less complete result with unfamiliar substitutions, and this experiment cannot tell us how safely the same rules would handle legitimate Unicode text in other languages, some possible next steps may be the study of smarter, less destructive and/or more generalized defenses for homoglyphs attacks.

Jev changes very little under these attacks, even when the local models lose much of their performance. The uncertainty is mainly about why it behaves this way and how far that behavior would generalize. Unknown training data, possible exposure to the datasets, and any preprocessing behind the API make it difficult to attribute the result to the model's architecture. Normalization similar to the defenses used here could explain part of its robustness, but I have no evidence that the API applies it. The experiment measures the behavior of the service as a whole.

7 Conclusions

I started this project to understand how changes that look minor or are invisible to a reader (like in the alt version of this report) can affect text classification. The experiments show that this is a practical weakness even for models that perform well on ordinary text. Character features offer a useful, simple baseline, and targeted preprocessing can recover much of the lost performance. Neither removes the need to test unfamiliar inputs.

Jev's stability is encouraging, but the limited visibility into its training and API leaves the explanation open. My main conclusion is that robustness should be evaluated across the whole system, including preprocessing, and across different kinds of changes. Before relying on a defense, I would also check that it preserves correct predictions on clean text and does not remove distinctions that matter for the task or language.

Use of GenAI and LLM tools. I used ChatGPT and Codex to brainstorm and refine the project design, locate and cross-check relevant literature and documentation, scaffold, draft initial versions and review notebooks, assist with debugging and statistical checks and writing tests, assist with reproducibility and seeding of the experiments, implement the extra jev study using the previous made code and the agent skill published by typesafeAI for the use of their API, revise the report's prose and nightmarish latex figures and table placement. The main controlled experiments use local classifiers. The Jev comparison uses separate calls to a paid hosted classification API. I remain responsible for running the experiments, verifying sources and results, and approving the submitted report and code.

References

[1] Steffen Eger, Gözde Gül Şahin, Andreas Rücklé, Ji-Ung Lee, Claudia Schulz, Mohsen Mesgar, Krishnkant Swarnkar, Edwin Simp-

son, and Iryna Gurevych. Text processing like humans do: Visually attacking and shielding NLP systems. In *Proceedings of NAACL-HLT*, pages 1634–1647, 2019. https://aclanthology.org/N19-1165/. 1, 2

[2] Nicholas Boucher, Ilia Shumailov, Ross Anderson, and Nicolas Papernot. Bad characters: Imperceptible NLP attacks. In *2022 IEEE Symposium on Security and Privacy*, pages 1987–2004, 2022. https://arxiv.org/abs/2106.09898. 2

[3] Renato Geh, Zilei Shao, and Guy Van Den Broeck. Adversarial tokenization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 20738–20765, 2025. https://aclanthology.org/2025.acl-long.1012/. 2

[4] Portia Cooper, Eduardo Blanco, and Mihai Surdeanu. The lies characters tell: Utilizing large language models to normalize adversarial Unicode perturbations. In *Findings of ACL*, pages 18932–18944, 2025. https://aclanthology.org/2025.findings-acl.969/. 2

[5] Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient intent detection with dual sentence encoders. In *Proceedings of the 2nd Workshop on NLP for Conversational AI*, pages 38–45, 2020. https://aclanthology.org/2020.nlp4convai-1.5/. 2

[6] Xin Li and Dan Roth. Learning question classifiers. In *COLING 2002: The 19th International Conference on Computational Linguistics*, 2002. https://aclanthology.org/C02-1150/. 2

[7] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019. https://arxiv.org/abs/1910.01108. 3

[8] Unicode Consortium. Unicode standard annex #15: Unicode normalization forms, 2026. https://unicode.org/reports/tr15/tr15-58.html. 3

[9] Unicode Consortium. Unicode technical standard #39: Unicode security mechanisms, 2026. https://unicode.org/reports/tr39/tr39-34.html. 3

[10] Diogo Almeida. Introducing System One models & Jev, September 2026. TypeSafe AI, 15 September 2026. Provider announcement. 8

[11] Nandakishor M. SalesRLAgent: A reinforcement learning approach for real-time sales conversion prediction and optimization. *arXiv preprint arXiv:2503.23303*, 2025. Submitted 30 March 2025. https://arxiv.org/abs/2503.23303. 8

[12] Convai Innovations. Laya: Public model weights and model card, 2026. Accessed 2 October 2026. https://huggingface.co/convaiinnovations/laya. 8

[13] TypeSafe AI. System One HTTP API reference, 2026. Provider documentation, accessed 30 September 2026. https://docs.typesafe.ai/api. 8

[14] TypeSafe AI. Models: Jev versions, pricing and limits, 2026. Provider documentation, accessed 30 September 2026. https://docs.typesafe.ai/models. 8